Conception d'API HTTP REST

Synthèse de révision complète — Séances 1 & 2 (cours + TP)

Module « une API par étudiant » — fil rouge — ESGI / CSGI

■ Ce document consolide le contenu magistral des slides **et** le détail des TP (gestes, contrats HTTP exacts, livrables, critères de réussite). L'examen et le contrôle continu portent sur les deux. Il complète les deux PDF de slides déjà fournis.

Repères transverses du module

- Objectif de maturité : **niveau 2 de Richardson partout** + une touche de niveau 3 (liens de pagination).
- 3 sujets possibles : **bibliothèque municipale**, **billetterie de cinéma**, **salle de sport** (un étudiant par sujet, gardé jusqu'à la soutenance en séance 10).
- Langage libre. Persistance non évaluée (store mémoire accepté).
- Chaque livrable est produit en séance et versionné (git).

Partie A — Socle conceptuel (à maîtriser pour l'écrit)

1. Le problème central : ressources, pas fonctions

Le mauvais réflexe est de coller une URL sur chaque fonction du code (`POST /createReservation`, `POST /getReservationsByUser`…). Chaque équipe invente alors ses noms et ses codes de retour : rien n'est prévisible, le client doit lire la doc pour chaque appel.

L'approche ressource : un **petit nombre de noms** (les ressources), un **petit nombre de verbes standard** (ceux d'HTTP), une **sémantique connue d'avance**.

2. Échelle de maturité de Richardson

Niveau	Ce qu'on ajoute	Ce que le client y gagne
0	HTTP sert de tunnel, tout en POST	rien
1	Des ressources, chacune son URI	une adresse stable par chose (désigner, relier, retrouver dans les logs)
2	Les verbes ET les codes HTTP employés sémantiquement	sémantique connue : GET se cache/rejoue, DELETE rejoué ne change rien, 4xx ≠
3	Hypermédia : liens vers la suite dans la réponse	le client suit les liens au lieu de fabriquer les URIs

■ **Piège d'examen :** un `POST` qui sert à lire, ou une erreur renvoyée en `200`, fait **redescendre au niveau 1**. Réf. : Leonard Richardson (2008), popularisé par Martin Fowler (2010).

3. Ressources et collections

- **Ressource** = une chose qu'on peut nommer (une réservation, un adhérent, une séance).
- **Collection** = une ressource qui contient d'autres ressources.

```
/reservations            la collection
/reservations/42         un élément de la collection
/seances/7/reservations  une sous-collection
```

■ On modélise **les choses, pas les actions**. « Annuler une réservation » n'est pas une ressource : c'est un changement d'état de `reservations/42`.

4. Concevoir les URIs — les règles

- **Noms au pluriel** pour les collections : `/ouvrages`, `/adherents`.
- **Pas de verbe dans l'URI** : `/reservations/42`, pas `/getReservation?id=42`.
- **Identifiant stable** dans le chemin.
- **Sous-ressource** quand la vie de l'une dépend de l'autre : `/ouvrages/128/exemplaires`.
- **Filtres, tri, pagination en query string**, jamais dans le chemin.

■ Le chemin identifie **une** ressource ou **une** collection. Tout ce qui restreint, ordonne ou pagine passe par la query string.

5. Verbes et leur sémantique

Verbe	Sûr	Idempotent	Usage
GET	oui	oui	lire une ressource ou une collection
POST	non	non	créer un élément dans une collection, déclencher un traitement
PUT	non	oui	remplacer une ressource entière à un emplacement connu
PATCH	non	non*	modifier partiellement (*idempotent seulement si conçu pour)
DELETE	non	oui	supprimer une ressource

- **Sûr** = ne modifie rien.
- **Idempotent** = rejouer la requête à l'identique laisse le serveur dans le même état qu'un seul appel.

Ces notions resservent en séance 4 pour la concurrence.

6. Codes de statut — familles et pièges

Famille	Sens	Exemples
2xx	succès	200 OK, 201 Created, 204 No Content
3xx	redirection / cache	304 Not Modified
4xx	le client s'est trompé	400, 401, 403, 404, 409, 422
5xx	le serveur a échoué	500, 502, 503

Les choix qui reviennent (souvent testés)

- `201 Created` après un POST réussi + en-tête `Location` vers la ressource créée.
- `204 No Content` quand il n'y a rien à renvoyer (souvent après DELETE).
- `400 Bad Request` : requête **mal formée** (JSON invalide, champ absent).
- `422 Unprocessable Content` : requête **bien formée mais sémantiquement invalide** (ex. date de fin avant date de début).
- `404 Not Found` vs `403 Forbidden` : « ça n'existe pas » vs « ça existe mais pas pour toi » — on renvoie parfois 404 exprès pour ne pas révéler l'existence.

■ Distinction 400 vs 422 et 404 vs 403 = pièges explicitement annoncés dans le cours.

7. Négociation de contenu

- `Content-Type` décrit le corps **qu'on envoie**.
- `Accept` décrit ce que le client **sait recevoir**.
- Le serveur choisit une représentation et l'annonce dans le `Content-Type` de la réponse.

■ Erreur classique : lire `Content-Type` sur une requête GET sans corps. C'est `Accept` qui porte la demande du client.

En-têtes conditionnels (ETag, If-Match, If-None-Match) : cache + concurrence, approfondis en séance 4.

8. Un échange complet de référence

```
POST /seances/7/reservations HTTP/1.1
Content-Type: application/json

{ "sieges": ["A12","A13"], "client": "c-901" }
```

```
HTTP/1.1 201 Created
Location: /reservations/42
Content-Type: application/json

{ "id":42, "seance":7, "sieges":["A12","A13"], "statut":"confirmee" }
```

Le client crée dans une collection (POST), le serveur répond **où** la ressource vit maintenant (Location). Aucune doc nécessaire pour comprendre l'échange.

Partie B — TP 1 : modèle de ressources + init du projet

Format : projet individuel, ~20 min guidé. Aucun code : uniquement conception + initialisation du dépôt.

Arborescence de départ

```
mon-api/
  README.md
  docs/
    api-design.md
  src/          (code, à partir de la séance 2)
  requests/     (collection de requêtes, à partir de la séance 2)
```

README — squelette

- Titre / nom de l'API.
- **Sujet fil rouge** : n° + nom (à remplir en fin de séance 1).
- **Stack** : langage / framework HTTP / client HTTP.
- **Lancer le projet** : commande unique (`docker compose up`) OU procédure pas à pas (deps, variables d'env, commande de démarrage, URL de base). Docker non exigé.

docs/api-design.md — les 3 sections attendues

(a) Ressources et collections — au moins 4 ressources LIÉES

Pour chacune : nom de collection au pluriel, 3 à 5 attributs, ses relations. Exemple :

```
ouvrages    (id, titre, auteurs[], isbn, annee)      -> a des exemplaires
exemplaires (id, ouvrage_id, etat, localisation)     -> sous-ressource d'ouvrages
adherents   (id, nom, email, actif)
emprunts    (id, exemplaire_id, adherent_id, du, au) -> lie exemplaire et adherent
```

(b) Tableau URI × verbe × code de statut — au moins 8 lignes

Opération	Méthode + URI	Succès	Erreurs prévues
Lister le catalogue	GET /ouvrages	200	—
Lire un ouvrage	GET /ouvrages/{id}	200	404
Créer un ouvrage	POST /ouvrages	201 + Location	400, 422
Lister les exemplaires	GET /ouvrages/{id}/exemplaires	200	404
Emprunter	POST /exemplaires/{id}/emprunts	201 + Location	404, 409

(c) 2 à 3 choix de conception justifiés (une phrase chacun)

Exemples fournis par le cours :

- « `emprunts` est une collection de premier niveau et non une sous-ressource de `adherents`, car un emprunt se consulte aussi côté exemplaire. »
- « Rendre un exemplaire = `PATCH /emprunts/{id}` qui renseigne `rendu_le`, plutôt qu'un `POST /exemplaires/{id}/retour` (verbe dans l'URI) ou qu'un `DELETE` (l'historique disparaîtrait). »

Contrats de référence agnostiques (sûr / idempotent)

Verbe	Sûr	Idempotent
GET	oui	oui
POST	non	non
PUT	non	oui
PATCH	non	non par défaut
DELETE	non	oui

Livrable & critères de réussite — TP 1

- Le dépôt se clone et le README explique comment lancer (même si rien ne tourne encore).
- `api-design.md` liste au moins 4 ressources **liées** entre elles.
- Chaque opération principale a un code de succès **et** ses erreurs prévues.
- Aucun verbe dans les URIs ; collections au pluriel.
- Design au **niveau 2 de Richardson** : chaque verbe employé pour sa sémantique.
- Sujet choisi + acté dans le README + premier commit poussé.

```
git init && git add -A && git commit -m "chore: init du depot, api-design, sujet choisi"
```

Partie C — TP 2 : CRUD et collection navigable

Format : ~15 min conception + remise du kit, puis ~37 min guidé. Le TP décrit des **gestes** et des **contrats HTTP**, pas des appels de framework. Store en mémoire accepté.

Étape schéma : modèle de données (esquisse)

Avant de coder, ajouter à `docs/api-design.md` une section courte (sans SQL complet) :

- tables et colonnes clés ;
- relations et cardinalités (1-N, N-N) ;
- ce qui est stocké vs ce qui est calculé.

■ Une ressource n'est pas une table : une table de liaison ne devient pas forcément une collection, et une info calculée (une disponibilité) n'a pas besoin de colonne.

Le kit de données (remis par le formateur)

Par sujet : `schema.sql` + `seed.sql` (SQLite), `seed.json` (mêmes données pour un store mémoire), et un README de chargement. Point de départ adaptable si l'esquisse diffère et qu'on sait le justifier.

Rappels de chargement (kit bibliothèque)

- SQLite n'active pas les FK par défaut : `PRAGMA foreign_keys = ON;` à chaque connexion.
- Dates en ISO 8601 (DATE = YYYY-MM-DD, TIMESTAMP UTC, TIME = HH:MM). « Maintenant » des données = lundi 2026-10-05.
- `emprunts.rendu_le = NULL` tant que non rendu. Auteurs multiples via table de liaison `ouvrages_auteurs`.

J1 — CRUD sur 2 ressources (contrats exacts)

```
POST   /ouvrages        -> 201 Created + Location: /ouvrages/128
GET    /ouvrages/128    -> 200 OK + la ressource
GET    /ouvrages        -> 200 OK + collection (paginee)
PUT    /ouvrages/128    -> 200 OK (remplacement complet) ou 204
PATCH  /ouvrages/128    -> 200 OK (champs fournis seulement)
DELETE /ouvrages/128    -> 204 No Content ; 404 si l'id n'existe pas
```

■ PUT **remplace** : un corps partiel efface les champs absents (bug fréquent). PATCH **modifie** les champs fournis.

■ DELETE sur un id inconnu -> **404**, pas 204.

Pas encore de validation fine ni de format d'erreur normalisé : c'est la séance 3 (RFC 9457, application/problem+json).

J2 — Rendre une collection navigable

Collection cible par sujet

Sujet	Collection navigable	Filtres attendus	Tri de départ
Bibliothèque	GET /ouvrages	auteur, disponible, annee_min	titre,id
Cinéma	GET /seances	film, date, salle	debut,id
Salle de sport	GET /creneaux	cours, coach, jour	debut,id

Pagination offset

```
GET /ouvrages?page=3&limit;=20        (equivaut a ?offset=40&limit;=20)
```

- Simple (LIMIT … OFFSET … en SQL), permet de sauter à la page N.
- **Dérive** si des éléments sont insérés/supprimés pendant la navigation (on peut sauter ou revoir un élément). La pagination par curseur corrige ça — séance 4.

Enveloppe de réponse — une forme unique, tenue sur TOUTES les collections

```
{
  "data": [ { "id":128, "titre":"Dune" } ],
  "pagination": { "page":3, "limit":20, "total":402 },
  "links": { "next":"/ouvrages?page=4&limit;=20", "prev":"/ouvrages?page=2&limit;=20" }
}
```

Filtrage

```
GET /ouvrages?auteur=herbert&disponible;=true&annee;_min=1960
```

- Chaque paramètre optionnel et combinable ; absent = pas de filtre.
- Paramètre inconnu : l'ignorer OU renvoyer 400 — **à décider et documenter**.

Tri stable — le piège

```
GET /ouvrages?tri=titre,id&ordre;=asc
```

■ Toujours une **clé de départage unique** (ex. `,id`). Sinon deux éléments de même valeur peuvent changer d'ordre entre deux appels → un élément apparaît page 2 puis page 3. Vérifier à la main.

Aparté cours — le problème N+1

Lister 20 ouvrages puis charger les exemplaires de chacun = 21 allers-retours. Remède : **2 requêtes** quelle que soit la taille de page (un `WHERE ... IN (...)`). Côté contrat : lien vers la relation, ou relation embarquée à la demande (`?include=exemplaires`). Pas de jointure naïve (LIMIT 20 compterait 20 lignes jointes, pas 20 ouvrages). Revu en séance 7 (GraphQL).

J3 — Collection de requêtes versionnée

```
requests/
  ouvrages-create
  ouvrages-get
  ouvrages-list
  ouvrages-list-filtered
  ouvrages-update
  ouvrages-delete
```

Client au choix (Bruno, Postman, Insomnia, Hoppscotch, curl). Exportée/écrite en fichiers puis commitée. Noter en **une phrase** la stratégie de pagination dans le README (« offset pour l'instant ; curseur en séance 4 »).

Livrable & critères de réussite — TP 2

- Section « Modèle de données (esquisse) » présente dans `api-design.md` ; kit chargé.
- API qui démarre et répond ; CRUD complet sur au moins 2 ressources, codes conformes.
- `POST` renvoie **201 ET un Location exploitable**.
- `DELETE` sur un id inconnu renvoie **404**.
- Collection navigable = `data` + métadonnées de pagination + liens.
- Tri avec clé de départage unique (tri stable vérifié à la main).
- `requests/` : au moins une requête par opération, commitée.
- Stratégie de pagination notée en une phrase dans le README.

Travail perso avant séance 3 : lire la fiche RFC 9457 (application/problem+json). Prochaines notions annoncées : format d'erreur normalisé (s3), concurrence + pagination curseur + en-têtes conditionnels (s4), relations/N+1 GraphQL (s7), soutenance (s10).